Amrutvahini College of Engineering, Sangamner Department of Electronics and Telecommunication Engineering
Experiment No.: 05

Date of Performance:

Aim: To Write VHDL code for Mod n counter.

Objective : To study Mod 4 counter

Software :- Quartus Prime Lite 24.1

Theory

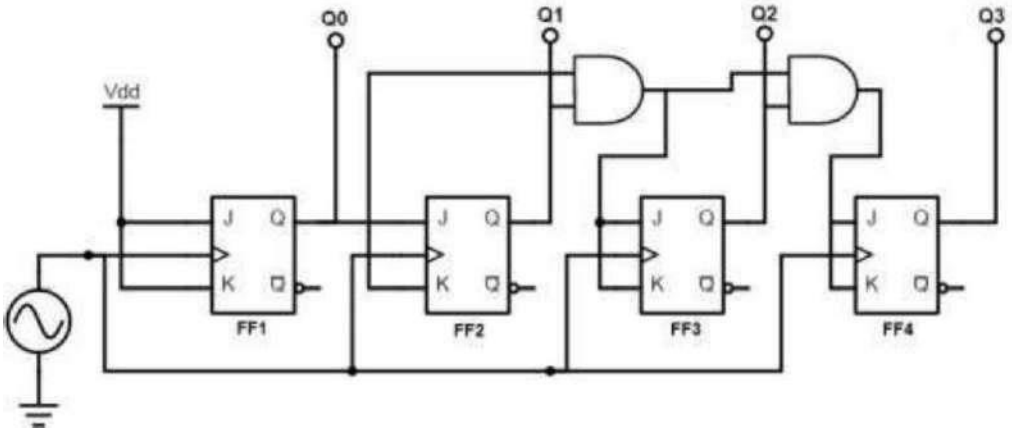
The counters which use clock signal to change their Transitions are called Synchronous Counters. This Means the synchronous counters depends on their clock input to change state values. In synchronous counters, all flip flops are connected to the same clock signal and all flip flops will trigger at the same time. Synchronous Counters are also known as "Simultaneous Counters". There is no Propagation delay and no ripple effect in synchronous counters.

4 bit Synchronous UP Counter

The 4 bit up counter shown in below diagram is designed by using JK flip flop. External clock pulse is connected to all the flip flops in parallel.

For designing the counters JK flip flop is preferred .The significance of using JK flip flop is that it can toggle its state if both the inputs are high, depending on the clock pulse.

The inputs of first flip flop are connected to HIGH (logic 1), which makes the flip flop to toggle, for every



clock pulse entered into it. So the synchronous counter will work with single clock signal and changes its state with each pulse. The output of first JK flip flop (Q) is connected to the input of second flip flop. The AND gates (which are connected externally) drives the inputs of other two flip flops. The inputs of these AND gates, are supplied from previous stage lip flop outputs. If inputs of FF2 are connected directly to the Q1 output of FF1, the counter would not function properly. This is because, the Q1 value is high at count of 210, and this means that the FF2 flip flop will toggle for the 3rd clock pulse. This results in wrong counting operation, gives the count as 710 instead of 410. To prevent this problem AND gates are used at the input side of FF2 and FF3. The output of the AND gate will be high only when the Q0, Q1 outputs are high. So for the next clock pulse, the count will be 00012.

Similarly, the flip flop FF3 will toggle for the fourth clock pulse when Q0, Q1 and Q2 are high. The Q3 output will not toggle till the 8th clock pulse and will again remain high until 16th clock pulse. After the 16th clock pulse, the q outputs of all flip flops will return to 0.

Operation

In the up counter the 4 bit binary sequence starts from 0000 and increments up to 1111.Before understanding the working of the above up counter circuit know about JK Flip flop.

In the above circuit as the two inputs of the flip flop are wired together. So, there are only two possible conditions that can occur, that is, either the two inputs are high or low.

If the two inputs are high then JK flip-flop toggles and if both are low JK flip flop remembers i.e. it stays in the previous state.

Let us see the operation. Here clock pulse indicates edge triggered clock pulse.

1.) In the first clock pulse, the outputs of all the flip flops will be at 0000.

2.) In the second clock pulse, as inputs of J and k are connected to the logic high, output of JK flip flop(FF0) change its state .Thus the output of the first flip-flop(FF0) changes its state for every clock pulse .This can be observed in the above shown sequence .The LSB changes its state alternatively. Thus producing -0001

3.) In the third clock pulse next flip flop (FF1) will receive its J K inputs i.e (logic high) and it changes its state. At this state FF0 will change its state to 0. And thus input on the FF1 is 0.Hence output is -0010

4.) Similarly, in the fourth clock pulse FF1 will not change its state as its inputs are in low state; it remains in its previous state. Though it produces the output to FF2, it will not change its state due to the presence of AND gate. FF0 will again toggle its output to logic high state. Thus Output is 0011.

5.) In the fifth clock pulse, FF2 receives the inputs and changes its state. While, FF0 will have low logic on its output and FF1 will also be low state producing 0100 input As it can go through 10 called as “Decade Counter”. A B C This process continuous up to 1111.

After reaching zero again the three flip flops toggles to logic low i.e 0000 and again count starts. Timing diagram for up counter is shown below.

Conclusion:

Questions:

1. What is a counter? What is a Mod-N counter? How do you determine the modulus of a counter?
2. What is the difference between an asynchronous and a synchronous counter?
3. How do you design a synchronous Mod-N counter?

VHDL CODE FOR MODNCOUNTER

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity mod_n_counter is
    Generic (
        N : integer := 10;          -- Modulo value (e.g., Mod-10)
        WIDTH : integer := 4        -- Number of bits required
    );
    Port (
        clk   : in  STD_LOGIC;
        reset : in  STD_LOGIC;
        q     : out STD_LOGIC_VECTOR(WIDTH - 1 downto 0)
    );
end mod_n_counter;

architecture Behavioral of mod_n_counter is
    signal count_reg : unsigned(WIDTH - 1 downto 0) := (others => '0');
begin
    process(clk, reset)
    begin
        if reset = '1' then
            count_reg <= (others => '0');
        elsif rising_edge(clk) then
            if count_reg = N - 1 then
                count_reg <= (others => '0');
            else
                count_reg <= count_reg + 1;
            end if;
        end if;
    end process;

    q <= std_logic_vector(count_reg);
end Behavioral;
```

TEST BENCH FOR MODNCOUNTER

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_mod_n_counter is
end tb_mod_n_counter;

architecture sim of tb_mod_n_counter is
    constant N_VAL : integer := 10;
    constant WIDTH_VAL : integer := 4;
    constant CLK_PERIOD : time := 10 ns;
```

```
    signal clk   : STD_LOGIC := '0';
    signal reset : STD_LOGIC := '1';
    signal q     : STD_LOGIC_VECTOR(WIDTH_VAL - 1 downto 0);
begin
    -- Instantiate the Unit Under Test (UUT)
    uut: entity work.mod_n_counter
        generic map (
            N => N_VAL,
            WIDTH => WIDTH_VAL
        )
        port map (
            clk   => clk,
            reset => reset,
            q     => q
        );

    -- Clock generation process
    clk_process: process
    begin
        clk <= '0';
        wait for CLK_PERIOD / 2;
        clk <= '1';
        wait for CLK_PERIOD / 2;
    end process;

    -- Stimulus process
    stim_process: process
    begin
        reset <= '1';
        wait for 20 ns;
        reset <= '0';
        wait for 150 ns; -- Allow counter to loop through cycles
        reset <= '1';    -- Test asynchronous/synchronous reset behavior
        wait for 20 ns;
        reset <= '0';
        wait;
    end process;
end sim;
```